

ECE 220 Computer Systems & Programming

Lecture 5 — Introduction to C: Data Types, Operators, Scope, Storage

September 8, 2026

Prof. Sayan Mitra mitras@illinois.edu

Course website: courses.grainger.illinois.edu/ece220/fa2026

I ILLINOIS

Electrical & Computer Engineering

GRAINGER COLLEGE OF ENGINEERING

First C: party.c

```
/* One program, every feature we need today */
#include <stdio.h>
#define TABLE 30          /* slices needed */

int total = 0;             /* global */

int main() {
    int pizzas = 3;        /* locals live */
    int each = 8;          /* on the stack */
    char room = 'A';
    int mine, enough;

    mine = pizzas * each + pizzas % 2;
    {
        int pizzas = 2;    /* TAs: two more */
        total = total + pizzas * each;
    }
    total = total + mine;
    enough = (total > TABLE); /* 0 or 1 */
    printf("%c%d: %d\n", room, enough, total);
    return 0;
}
```

First C: party.c

```
/* One program, every feature we need today */
#include <stdio.h>
#define TABLE 30          /* slices needed */

int total = 0;             /* global */

int main() {
    int pizzas = 3;        /* locals live */
    int each = 8;          /* on the stack */
    char room = 'A';
    int mine, enough;

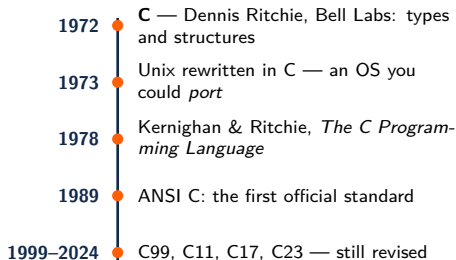
    mine = pizzas * each + pizzas % 2;
    {
        int pizzas = 2;    /* TAs: two more */
        total = total + pizzas * each;
    }
    total = total + mine;
    enough = (total > TABLE); /* 0 or 1 */
    printf("%c%d: %d\n", room, enough, total);
    return 0;
}
```

It compiles, it runs, it prints

A1: 41

By the end of today you will know what each line *becomes*: where every variable lives, and which instructions do the work.

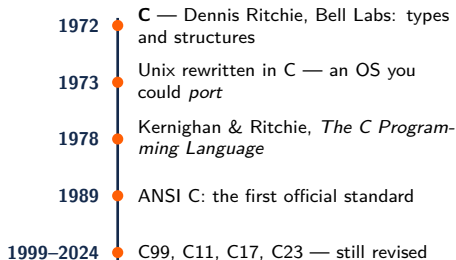
Where C Came From



C in the wild

- **Mars rovers** — Curiosity and Perseverance flight software
- **Linux kernel** — most servers, every Android phone
- **CPython** — the Python you use is a C program
- **SQLite, Git, nginx, FFmpeg**

Where C Came From



C in the wild

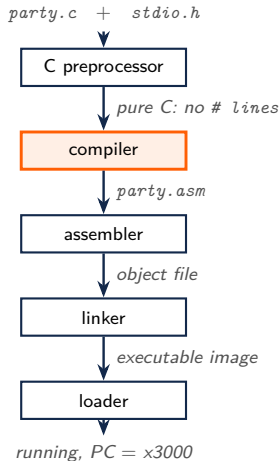
- **Mars rovers** — Curiosity and Perseverance flight software
- **Linux kernel** — most servers, every Android phone
- **CPython** — the Python you use is a C program
- **SQLite, Git, nginx, FFmpeg**

Why it stuck

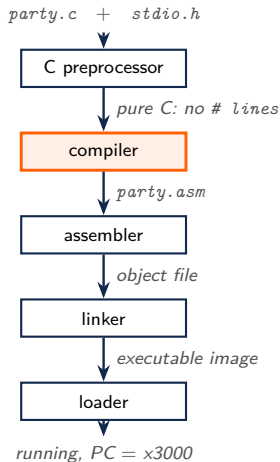
- Built to replace assembly language: write the operating system once, move it to a new machine by writing a new *compiler*
- Close to the hardware — every construct is a handful of instructions, exactly what we are doing by hand today
- Ritchie and Thompson received the ACM **Turing Award** in 1983
- Its syntax lives on in C++, Java, C#, Go, Rust
- The language is permissive, so serious users bolt discipline on top: MISRA C for automotive code, JPL's own C standard for flight software

Dates: D. M. Ritchie, HOPL-II, 1993; Kernighan & Ritchie, *The C Programming Language*, 1978. Coding rules: G. J. Holzmann, IEEE Computer, 2006.

From Source to Running Code



From Source to Running Code



- **Preprocessor** — text surgery only: `#include` pastes the header in, `#define` substitutes. Output is still C.
- **Compiler** — C to LC-3 assembly. **This is the box we open today.**
- **Assembler** — assembly to an object file of 1s and 0s.
- **Linker** — joins your object file with the library's, where `printf` already sits, into one image.
- **Loader** — copies that image into memory and sets the PC.

In which stage does `TABLE` disappear? the preprocessor
One `gcc` command hides all five programs.

Today's Question

**The machine has not changed.
It still only knows ADD, AND, LD, ST, BR.**



So: what does a *C declaration*, *expression*, and *block* each become down there? Four

features carry the whole translation: data types, storage, operators, scope. We take them one at a time.

Declarations: Type, Name, Home

```
int total = 0;           /* global */

int main() {
    int pizzas = 3;       /* locals live */
    int each = 8;         /* on the stack */
    char room = 'A';
    int mine, enough;
```

Every variable has a **type** (how many bits, read how), an **identifier**, and a **home** in memory.

- `int` — a 16-bit word on the LC-3
- `char` — one character; `'A'` is the number x0041
- `float`, `double` — no LC-3 hardware for these

Declarations: Type, Name, Home

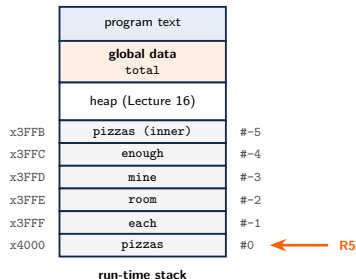
```
int total = 0;           /* global */

int main() {
    int pizzas = 3;      /* locals live */
    int each = 8;        /* on the stack */
    char room = 'A';
    int mine, enough;
```

Every variable has a **type** (how many bits, read how), an **identifier**, and a **home** in memory.

- **int** — a 16-bit word on the LC-3
- **char** — one character; 'A' is the number x0041
- **float**, **double** — no LC-3 hardware for these

Two different homes:



Storage class answers *how long it lives*:

- **static** (globals): the whole run — one fixed address, like a **.FILL** you already know
- **automatic** (locals): born at block entry, gone at exit

Declarations, Compiled

```
#define TABLE 30          /* slices needed */

int total = 0;              /* global */

int main() {
    int pizzas = 3;         /* locals live */
    int each = 8;           /* on the stack */
    char room = 'A';
    int mine, enough;
```

- `total` gets a *label*: a fixed word of memory
- The five locals sit together, each at an *offset* from one pointer, `R5`
- `TABLE` gets no home: `#define` is textual substitution
- Where `R5` comes from — a frame on the run-time stack — is Lecture 8

```
.ORIG x3000

; int pizzas = 3;  int each = 8;

; char room = 'A';  ('A' is just x0041)
```

Declarations, Compiled

```
#define TABLE 30          /* slices needed */

int total = 0;              /* global */

int main() {
    int pizzas = 3;         /* locals live */
    int each = 8;           /* on the stack */
    char room = 'A';
    int mine, enough;
```

- `total` gets a *label*: a fixed word of memory
- The five locals sit together, each at an *offset* from one pointer, `R5`
- `TABLE` gets no home: `#define` is textual substitution
- Where `R5` comes from — a frame on the run-time stack — is Lecture 8

```
.ORIG x3000
MAIN    LD    R5, LOCALS      ; base of the locals
; int pizzas = 3; int each = 8;
        AND   R0, R0, #0
        ADD   R0, R0, #3
        STR   R0, R5, #0
        AND   R0, R0, #0
        ADD   R0, R0, #8
        STR   R0, R5, #-1
; char room = 'A'; ('A' is just x0041)
        LD    R0, CHA
        STR   R0, R5, #-2
```

Local `pizzas` is now `LDR R0, R5, #0` — no name survives compilation, only an offset.

One Expression, Six Instructions

```
mine = pizzas * each  
      + pizzas % 2;
```

- **Precedence:** * and % bind tighter than +: (pizzas*each) + (pizzas%2)
- Both operands are *variables*, so the compiler emits JSR MULT
- MULT returns R2 and leaves R0 alone: pizzas needs no reload
- % 2 on a non-negative int is AND with 1

```
; mine = pizzas * each ...
```

```
; ... + pizzas % 2;
```

One Expression, Six Instructions

```
mine = pizzas * each  
      + pizzas % 2;
```

- **Precedence:** `*` and `%` bind tighter than `+`: `(pizzas*each) + (pizzas%2)`
- Both operands are *variables*, so the compiler emits JSR MULT
- `MULT` returns `R2` and leaves `R0` alone: `pizzas` needs no reload
- `% 2` on a non-negative int is AND with 1

```
; mine = pizzas * each ...  
    LDR R0, R5, #0      ; pizzas  
    LDR R1, R5, #-1     ; each  
    JSR MULT            ; R2 = R0 * R1  
; ... + pizzas % 2;  
    AND R0, R0, #1      ; R0 survived MULT  
    ADD R2, R2, R0  
    STR R2, R5, #-3     ; mine
```

With `pizzas = 3`, `each = 8`: $24 + 1 = 25$.

Operators You Will Use Today

Arithmetic: `+` `-` `*` `/` `%`

Relational: `==` `!=` `<` `<=` `>` `>=`

A comparison *evaluates to a value*: 1 if true, 0 if false.

That is why `enough = (total > TABLE);` is legal.

Logical: `&&` `||` `!` **Bitwise:** `&` `|` `^` `~` `«`

`»`

Assignment: `=` — and its evil twin `==`. `if (x = 3)` compiles, assigns, and is always true.

Operators You Will Use Today

Arithmetic: `+` `-` `*` `/` `%`

Relational: `==` `!=` `<` `<=` `>` `>=`

A comparison *evaluates to a value*: 1 if true, 0 if false.

That is why `enough = (total > TABLE);` is legal.

Logical: `&&` `||` `!` **Bitwise:** `&` `|` `^` `~` `«`

`»`

Assignment: `=` — and its evil twin `==`. `if (x = 3)` compiles, assigns, and is always true.

In-class — with `pizzas = 3`:

- `pizzas / 2 = 1 (integer!)`
- `pizzas % 2 = 1`
- `pizzas « 2 = 12`
- `pizzas > 30 = 0`
- `!(pizzas) = 0`

Integer division truncates — `5/9` is 0, not 0.55.
This bites everyone once.

Two Variables Named `pizzas`

```
{  
    int pizzas = 2;  /* TAs: two more */  
    total = total + pizzas * each;  
}  
total = total + mine;
```

- **Scope** = the region of *text* where a name is visible
- The inner `pizzas` **shadows** the outer one until the closing brace
- After the block, `pizzas` is 3 again — the inner one is gone
- `total` is global: visible everywhere, alive throughout

```
; { int pizzas = 2;  total = total + pizzas * each; }
```

Two Variables Named `pizzas`

```
{
    int pizzas = 2; /* TAs: two more */
    total = total + pizzas * each;
}
total = total + mine;
```

- **Scope** = the region of *text* where a name is visible
- The inner `pizzas` **shadows** the outer one until the closing brace
- After the block, `pizzas` is 3 again — the inner one is gone
- `total` is global: visible everywhere, alive throughout

```
; { int pizzas = 2; total = total + pizzas * each; }
AND R0, R0, #0
ADD R0, R0, #2
STR R0, R5, #-5 ; the INNER pizzas
LDR R1, R5, #-1 ; each
JSR MULT ; R2 = 2 * each
LD R0, TOTAL
ADD R0, R0, R2
ST R0, TOTAL
```

Compiled, the confusion vanishes: two names, two *different offsets* (`#0` and `#-5`). The compiler never mixes them up — only humans do.

Global access is LD/ST by label; local access is LDR/STR by offset from R5.

Finish the Compilation

Two lines of `party.c` are still uncompiled:

```
mine = ... + pizzas % 2;
```

One instruction. Which one, and why does it work only for non-negative `pizzas`?

```
enough = (total > TABLE);
```

No compare instruction: subtract, then branch. Set `enough` to 0 or 1 — four instructions.

AND `R1,R1,#1` keeps the low bit, which is the remainder mod 2. For negatives, C truncates toward zero, so $-3\%2 = -1$ and the trick fails.

```
; ... + pizzas % 2;  
; enough = (total > TABLE);
```

Finish the Compilation

Two lines of `party.c` are still uncompiled:

```
mine = ... + pizzas % 2;
```

One instruction. Which one, and why does it work only for non-negative `pizzas`?

```
enough = (total > TABLE);
```

No compare instruction: subtract, then branch. Set `enough` to 0 or 1 — four instructions.

`AND R1,R1,#1` keeps the low bit, which is the remainder mod 2. For negatives, C truncates toward zero, so $-3\%2 = -1$ and the trick fails.

```
; ... + pizzas % 2;
    AND R0, R0, #1      ; R0 survived MULT
    ADD R2, R2, R0
    STR R2, R5, #-3     ; mine
; enough = (total > TABLE);
    AND R1, R1, #0      ; enough = 0
    LD  R2, NEGTLB      ; -30
    ADD R2, R0, R2       ; total - 30
    BRnz ENDONE         ; <= 30: stay 0
    ADD R1, R1, #1      ; enough = 1
ENDONE STR R1, R5, #-4
```

The Whole Translation

```

/* One program, every feature we need today */
#include <stdio.h>
#define TABLE 30      /* slices needed */

int total = 0;          /* global */

int main() {
    int pizzas = 3;      /* locals live */
    int each = 8;        /* on the stack */
    char room = 'A';
    int mine, enough;

    mine = pizzas * each + pizzas % 2;
    {
        int pizzas = 2; /* TAs: two more */
        total = total + pizzas * each;
    }
    total = total + mine;
    enough = (total > TABLE); /* 0 or 1 */
    printf("%c%d: %d\n", room, enough, total);
    return 0;
}

```

```

.ORIG x3000
MAIN    LD    R5, LOCALS      ; base of the locals
; int pizzas = 3; int each = 8;
        AND   R0, R0, #0
        ADD   R0, R0, #3
        STR   R0, R5, #0
        AND   R0, R0, #0
        ADD   R0, R0, #8
        STR   R0, R5, #-1
; char room = 'A'; ('A' is just x0041)
        LD    R0, CHA
        STR   R0, R5, #-2
; mine = pizzas * each ...
        LDR   R0, R5, #0      ; pizzas
        LDR   R1, R5, #-1     ; each
        JSR   MULT            ; R2 = R0 * R1
; ... + pizzas % 2;
        AND   R0, R0, #1      ; R0 survived MULT
        ADD   R2, R2, R0
        STR   R2, R5, #-3     ; mine

```

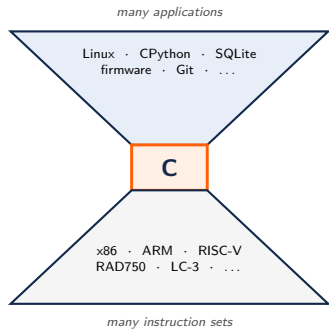
```

; { int pizzas = 2; total = total + pizzas * each
; }
        AND   R0, R0, #0
        ADD   R0, R0, #2
        STR   R0, R5, #-5     ; the INNER pizzas
        LDR   R1, R5, #-1     ; each
        JSR   MULT            ; R2 = 2 * each
        LD    R0, TOTAL
        ADD   R0, R0, R2
        ST    R0, TOTAL
; total = total + mine;
        LD    R0, TOTAL
        LDR   R1, R5, #-3
        ADD   R0, R0, R1
        ST    R0, TOTAL
; enough = (total > TABLE);
        AND   R1, R1, #0      ; enough = 0
        LD    R2, NEGTL      ; -30
        ADD   R2, R0, R2      ; total - 30
        BRnz  ENDONE         ; <= 30: stay 0
        ADD   R1, R1, #1      ; enough = 1
ENDONE   STR   R1, R5, #-4
; printf: a library call -- Thursday
        HALT

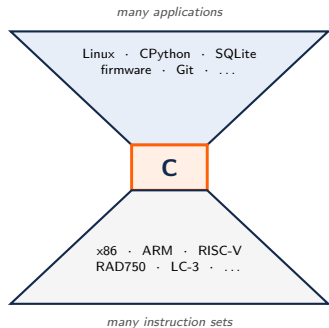
```

Every C construct became instructions you already knew. The compiler is a very fast, very literal version of what we just did by hand.

Why C Lasts: the Narrow Waist



Why C Lasts: the Narrow Waist



The same shape elsewhere



The genetic code — 64 codons, 20 amino acids, the same in nearly every living thing



Money — ends barter's double coincidence of wants



The grid — any source, any load, one 120 V / 60 Hz standard

$N \times M$ becomes $N + M$: each machine implements C once, each program targets C once.

The price of a waist is that it ossifies — everything depends on it. And the instruction set below is a waist too: you have been standing on it since Lecture 1.

Summary & What's Next

Today: one program, compiled by hand

- **Data types** say how many bits and how to read them; `char` is just a small integer
- **Storage** says where and for how long: globals get a fixed label, locals get offsets from `R5`; how `R5` itself is set up is Lecture 8
- **Operators** become `ADD`, `AND`, shifts — or a `JSR` when the machine has no instruction at all, as for `*`; precedence decides the order, and a comparison produces a 0/1 *value*
- **Scope** is a compile-time matter — shadowed names are simply different offsets

Next lecture

- The one line we skipped: `printf` — plus `if`, loops, and `scanf`

Code from today: `party.c` and its hand compilation `party.asm`. Run the C, trace the assembly, convince yourself they agree.